

Homework 3

Autonomous Mobile Robots

Sravanth Chowdary Potluri und5uv

November 15, 2024

Instructions And Inferences

This document provides the results of the codes of the assignment provided in the Codes folder. The file names are appropriately labelled to indicate the question they are presenting the solution for. In the Kalman filter code, the point robot starts at (50,50) in the workspace and goes to a randomly selected point at constant velocity. It is also assumed that the Robot internally knows the correct position and direction to the goal and converges upon it at constant speed. You can clearly see from the graphs that the kalman filter estimate is much better than the sensor measurements and that the estimate gets better for both position and velocity. The distance to goal over time plot is generated by the distance of the kalman estimate to the goal, You can see the noise in it, while the ideal estimate would be a perfect straight line. For the particle filter you can see by running the code that the particles converge to the True position of the robot as the steps go on and that the position estimation error over time and the orientation estimation error over time converge to zero demonstrating that the filter works. Animation for the particle filter can be displayed by running the code attached. For the Extra credit part, the particle convergence can be achieved by tuning the parameters such as robot movement better and letting the code run for more steps. The plots shown in the solution are in the case when the particles do a good job of converging to the True robot position, but please note that this does not happen always because the robot has only limited sensor range meaning it's unable to localize accurately all the time in the given situation. Also Please do wait until the initial animation is complete, Otherwise, there might be some error with the generation of other plots.

Question 1

Kalman Filter (50pt): Create a 100mx100m environment. Simulate a go-to-goal behavior with constant cruise control speed between two points, 50m apart. Implement a simple point mass robot. The robot has 2 GPS sensors (s1 and s2) estimating its position (i.e., each sensor return x-y position of the robot). The sensor errors are modeled as 0 mean Gaussian distributions

- For s1 $\mu_1x = \mu_1y = N(0, 6)$
- For s2 $\mu_2x = \mu_2y = N(0, 4)$

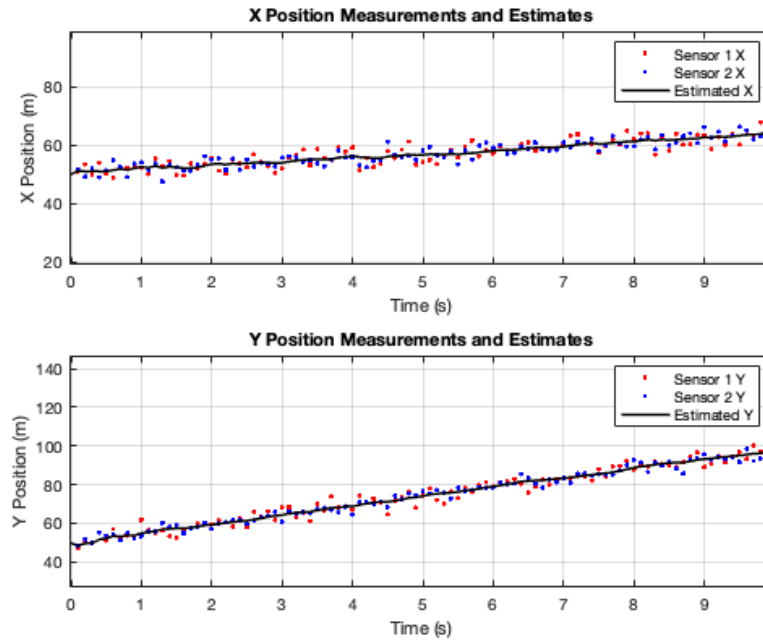


Figure 1: X and Y Position Sensor Measurements and Estimate Vs Time

- The input u to the robot has noise modeled as 0 mean Gaussian $\gamma = N(0, 1)$
- Implement a Kalman Filter to estimate the state of the system as it is running to the goal. Estimate both the position and the velocity of the robot.
- Plot estimated robot position (x,y) , linear velocity, and distance to the goal over time.
- On another figure plot each sensor measurement together with the estimate from the filter. You should see that the estimate is better than the measurements

Solution

Plots attached

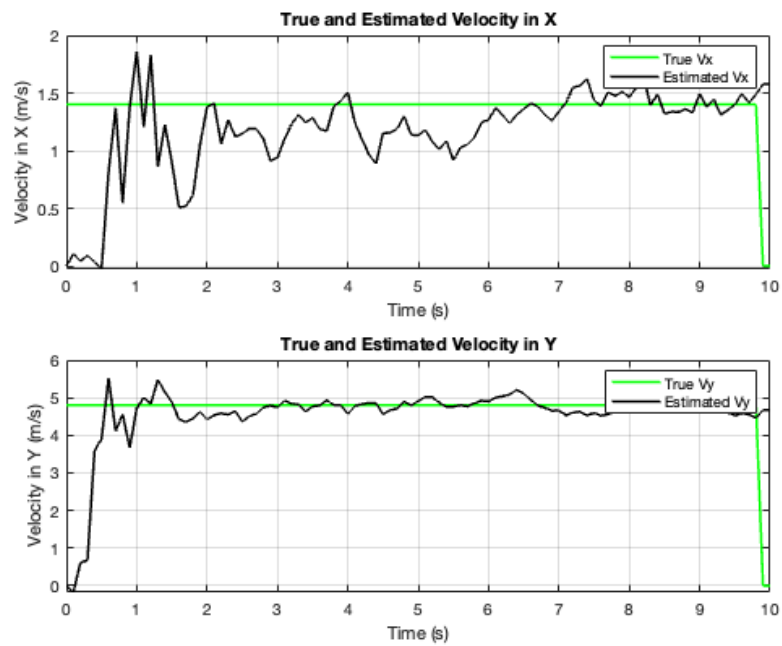


Figure 2: X and Y Velocity Estimates and True Velocities Vs Time

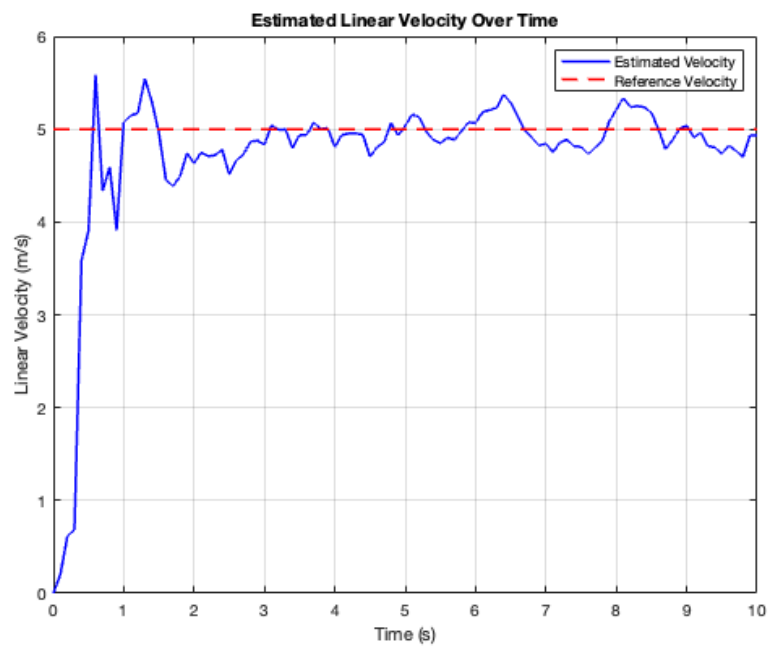


Figure 3: Magnitude Of Velocity Estimate and Reference Velocity Vs Time

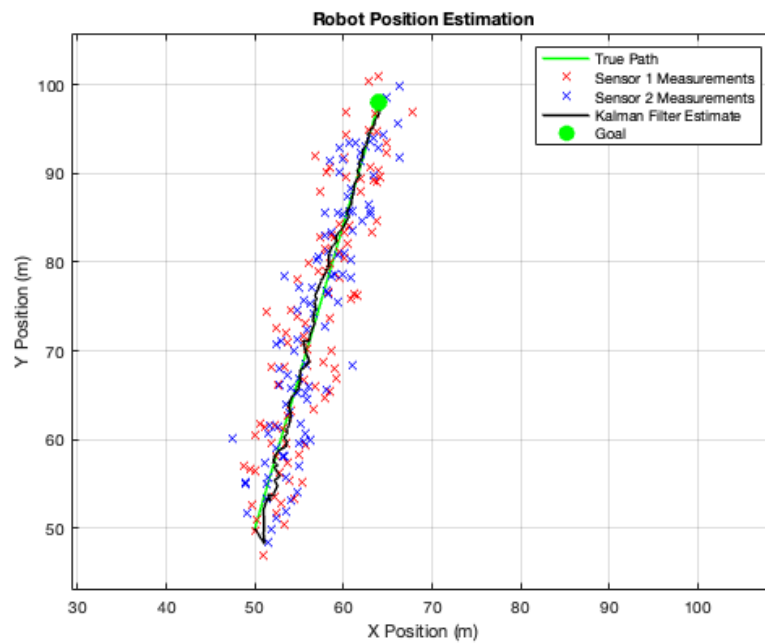


Figure 4: Robot Sensor Measurements, Kalman Filter Estimate and True Pat to Goal

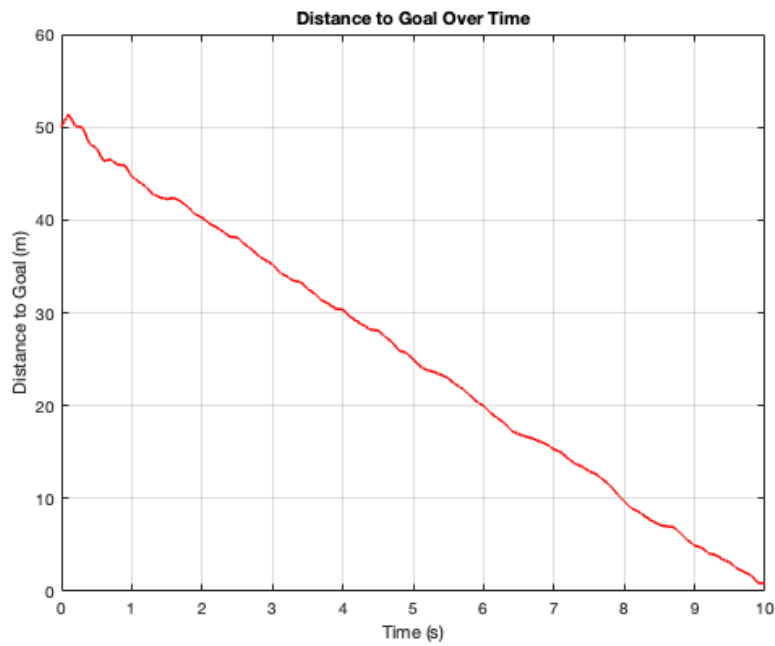


Figure 5: Distance To Goal Over Time

Question 2

Particle Filter (50pt): Create a 100mx100m environment. Create 6 landmarks at the following locations (25,25)(25,70)(70,25)(70,70)(10,40)(80,60)

- Place your robot in a random position and orientation in the middle of the environment
- Guess an initial pose of your robot in the environment
- Create 1000 particles scattered uniformly through the environment. Each particle should contain (random) x,y position and orientation.
- The robot can measure the distance to each landmark with some error $\mu = N(0, 8)$.
- The robot forward and turning motion noise is $\gamma = N(0, 0.5)$
- Note that these errors apply also to the particles!
- Implement a particle filter to localize the robot after 10 steps. In particular pay attention on implementing
 - a weight function as shown in class
 - the resampling procedure as shown in class
- NOTE: at each step make the robot move forward and rotate. It's up to you to decide the control input but the robot should not jump around. You can try with "move forward 1m" and "rotate 0.2 rad" at every step.
- Print the 10 estimated poses of the robot and the error between the actual pose and each estimate. HINT: average all particle values.

Solution

Plots attached

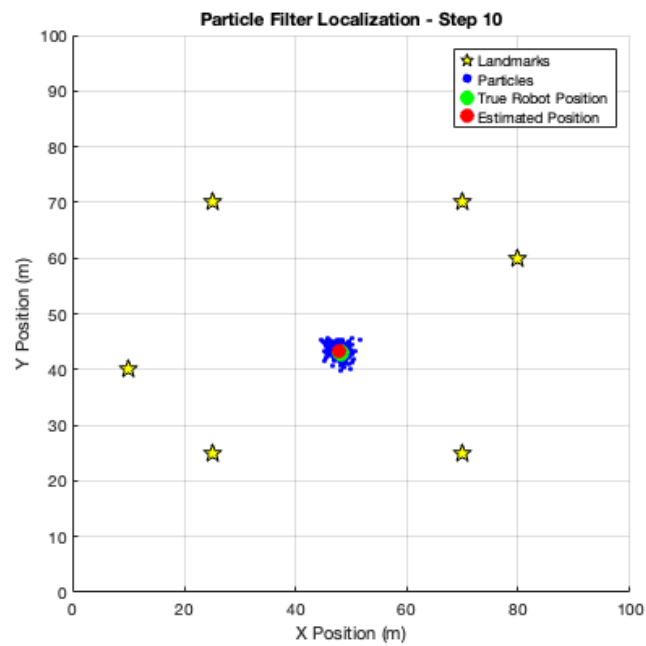


Figure 6: Enter Caption

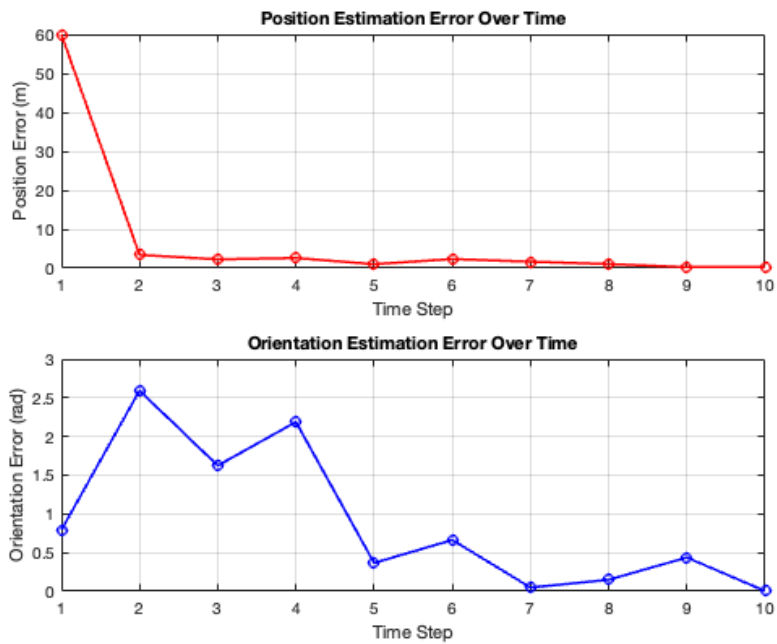


Figure 7: Estimation Errors Of Position and Orientation Over Time

Extra Credit

re-implement everything in Particle Filter considering that your sensor has limited range = 30m

Solution

Plots attached

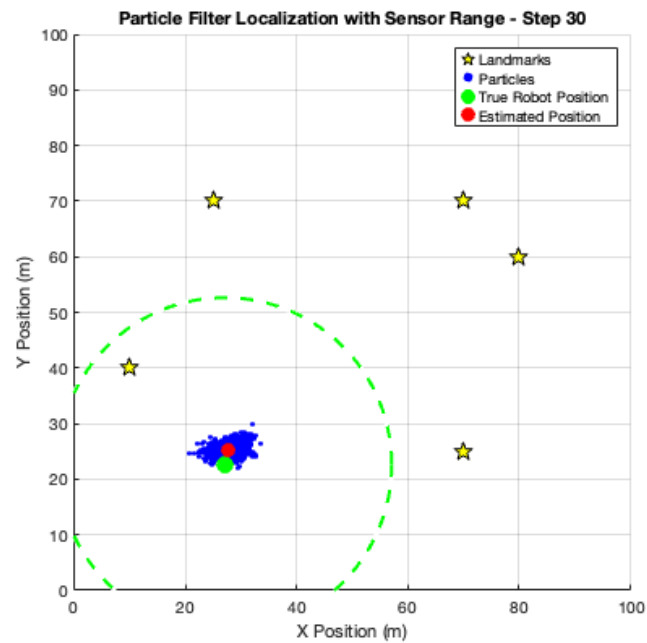


Figure 8: Final State of the Particle Filter, True Robot Position and Estimated Position With Limited Sensor Range

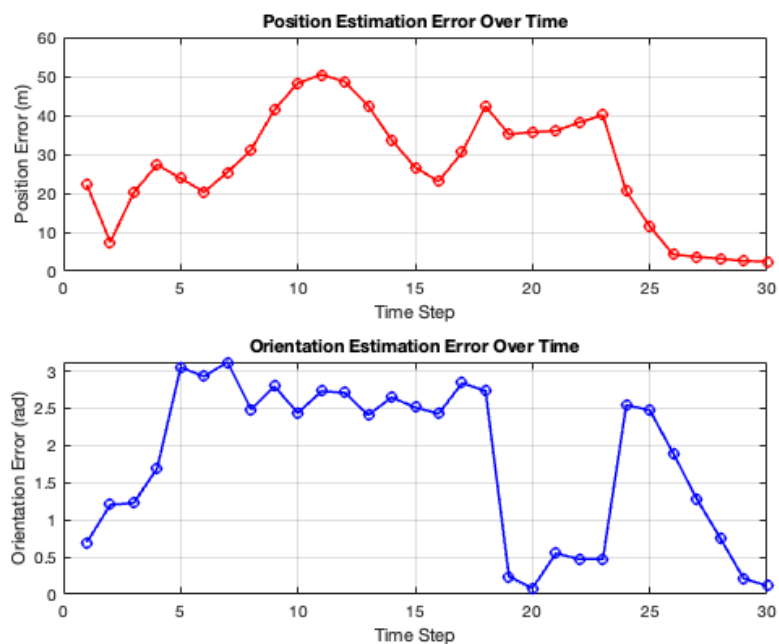


Figure 9: Estimation Errors Of Position and Orientation Over Time With Limited Sensor Range